



# SHEARED LLAMA: ACCELERATING LANGUAGE MODEL PRE-TRAINING VIA STRUCTURED PRUNING

**YeJe Choi**

---

2025.06.11

On-Device AI: ON THE Air

---

## Contents

# 1

---

서론

- 1. 연구 배경
  - 2. 연구 가설
- 

# 2

---

핵심  
방법론

- 1. LLM-SHEARING
- 

# 3

---

실험 및  
결과

- 1. EXPERIMENTS
-

# 1. 서론

## 연구 배경 및 연구 가설

이 논문은 이 문제에 대한 근본적인 질문에서 시작,

“이미 잘 학습된 대형 LLM을 활용해서, 훨씬 적은 컴퓨팅 리소스로 더 작고 경쟁력 있는 **범용 LLM**을 만들 수

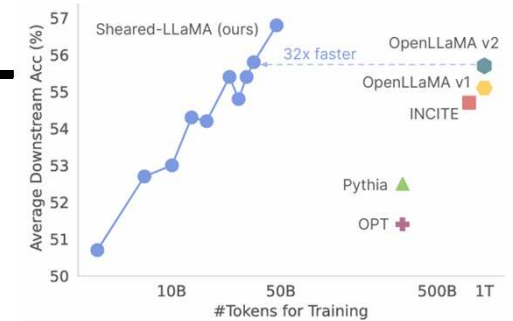


Figure 1: Sheared-LLaMA-2.7B surpasses a series of open-source models at a similar scale and only requires 1/32 (3%) of budget to achieve on-par performance with OpenLLaMA-3B-v2.

- 기존 가지치기(Pruning) 연구는 주로 특정 테스트(task-specific)에 맞게 모델을 압축하여 추론 속도를 높이는 데 집중함
- 하지만, 이 방법을 범용 LLM에 적용하면 성능 저하가 불가피하며, 손실된 성능을 회복하기 위한 사후 학습(Post-pruning)에 대한 연구도 부족한 상황



Sheared LLaMA는 가지치기를 ‘모델 압축’이 아닌 ‘효율적인 소형 모델 생성 방법론’으로 재정하고,  
이 과정을 최적화하는 새로운 프레임워크인 LLM-Shearing을 제안

---

## 2. 핵심 방법론

“LLM-SHEARING” ← Sheared LLaMA 논문에서 제시된 효율적인 소형 언어 모델(Large Language Model, LLM) 생성 방법론

**LLM-SHEARING** 방법론: 대형 사전 훈련된 모델을 활용하여 구조적 가지치기(structured pruning)와 계속된 훈련(continued pre-training)을 결합하여 더 작은 크기이면서도 경쟁력 있는 성능을 유지하는 모델을 만드는 방식

**LLM-SHEARING**의 목표는 더 작은 모델을 만들되, 성능 저하를 최소화하고, 적은 컴퓨팅 자원을 사용하여 효율적인 훈련을 가능하게 하는 것



1. 목표 지향적 구조적 가지치기(Targeted Structured Pruning)
  2. 동적 배치 로딩(Dynamic Batch Loading)
-

---

## 2. 핵심 방법론

### “LLM-SHEARING”: 목표 지향적 구조적 가지치기(Targeted Structured Pruning)

원본 대형 모델에서 목표 아키텍처로 가지치기를 수행하는 과정

목표는 모델을 더 작은 크기로 만들기 위해 **모델 구조**(layer, attention heads, hidden dimensions 등)를 최적화하는 것

이 과정에서 성능 손실을 최소화하면서도 필요한 모델 크기에 도달하도록 최적화 **필요한 모델 크기**에 도달하도록 최적화 됨

- 원본 모델에서 **목표 아키텍처**에 맞게 **불필요한 부분**을 가지치기하여 모델 크기를 압축
  - 가지치기 **마스크**를 학습하고, **라그랑주 승수**와 같은 수학적 기법을 사용하여 **최적화**
  - **제약 최적화 문제**를 풀어서 원하는 **모델 아키텍처**를 얻음
-

## Targeted Structured Pruning 설명

1. 목표 아키텍처 설정:
  - 모델의 목표 크기와 구조(레이어 수, 어텐션 헤드 수, 히든 차원 수 등)를 미리 정의
2. 프루닝 마스크 학습:
  - 마스크(mask)를 학습하여 어떤 구조를 유지하고, 어떤 구조를 제거할지 결정
3. 최적화:
  - 제약 최적화를 통해 성능 손실을 최소화하면서 목표 아키텍처에 맞는 모델을 만듦

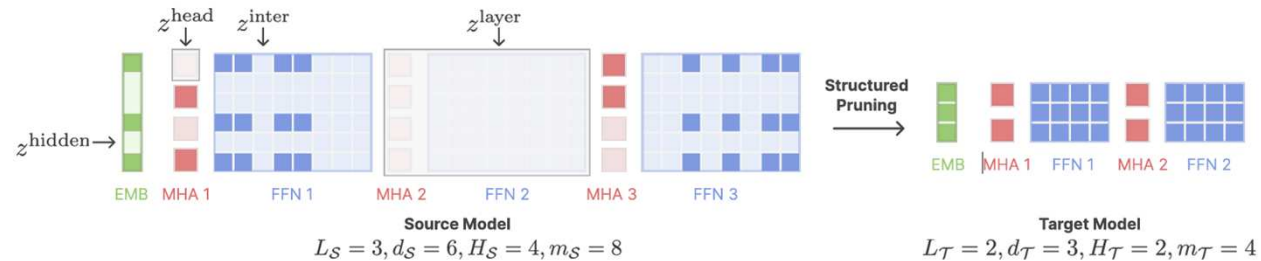


Figure 2: *Targeted structured pruning* produces a compact and dense model of a pre-specified shape. Light colors indicate pruned substructures. Masking variables  $z$  are learned to control whether a substructure is pruned ( $z = 0$ ) or retained ( $z = 1$ ).

| Granularity   | Layer                                   | Hidden dimension                         | Head                                                | Intermediate dimension                             |
|---------------|-----------------------------------------|------------------------------------------|-----------------------------------------------------|----------------------------------------------------|
| Pruning masks | $z^{\text{layer}} \in \mathbb{R}^{L_S}$ | $z^{\text{hidden}} \in \mathbb{R}^{d_S}$ | $z^{\text{head}} \in \mathbb{R}^{H_S} (\times L_S)$ | $z^{\text{int}} \in \mathbb{R}^{m_S} (\times L_S)$ |

\_\_\_\_\_

rectly. For example, for a target number of heads  $H_{\mathcal{T}}$  (and we use  $L_{\mathcal{T}}$ ,  $d_{\mathcal{T}}$ , and  $m_{\mathcal{T}}$  to represent the target number of layers, hidden dimension, and intermediate dimension respectively), we have the imposed constraint on a single layer as:

$$H_{\mathcal{T}} \leq \frac{L_{\mathcal{T}}}{L} \cdot \frac{d_{\mathcal{T}}}{d} \cdot \frac{m_{\mathcal{T}}}{m} \quad (1)$$

$$\tilde{\mathcal{L}}^{\text{head}}(\lambda, \phi, z) = \lambda^{\text{head}} \cdot \left( \sum z^{\text{head}} - H_{\mathcal{T}} \right) + \phi^{\text{head}} \cdot \left( \sum z^{\text{head}} - H_{\mathcal{T}} \right)^2.$$

첫 번째 항은 어텐션 헤드 수가 목표 값  $H$  과 얼마나 차이나는지 측정,  $\rightarrow$  그 차이를 최소화

두 번째 항은 첫 번째 항에 추가적인 제약을 부과, 위의 차이가 클수록 더 큰 패널티를 부여

Similar constraints are applied to pruning other substructures. Overall, we jointly optimize the model weights and pruning masks by a min-max objective  $\min_{\theta, z} \max_{\lambda, \phi} \mathcal{L}_{\text{prune}}(\theta, z, \lambda, \phi)$ :

$$\mathcal{L}_{\text{prune}}(\theta, z, \lambda, \phi) = \underbrace{\mathcal{L}(\theta, z)}_{\text{pruning loss}} + \sum_{j=1}^{L_S} \tilde{\mathcal{L}}_j^{\text{head}} + \sum_{j=1}^{L_S} \tilde{\mathcal{L}}_j^{\text{int}} + \tilde{\mathcal{L}}^{\text{layer}} + \tilde{\mathcal{L}}^{\text{hidden}},$$

마스크가 적용된 모델의 원래 언어 모델링 손실

$j-1$  ← 최대화  
 각 구조(헤드, 중간 차원 등)가 목표 개수와 일치하도록 강제하는 제약 조건

이 손실 함수를 min-max 최적화를 통해 학습하면, 모델은 원래 성능을 최대한 유지하면서 우리가 원하는 목표 구조로 점차 변형됨

---

## 2. 핵심 방법론

### “LLM-SHEARING”: 동적 배치 로딩(Dynamic Batch Loading)

가지치기를 통해 작은 모델을 얻은 후에 손실된 성능을 회복하기 위해 추가적인 사전 학습(continued pre-training)을 수행해야 함.

이때 기존 데이터 분포를 그대로 사용하면 비효율적이므로, **동적 배치 로딩** 기법을 사용하여 **각 도메인에서 학습이 더 필요한 부분을 집중적으로 학습**시킴.

- **계속된 훈련**을 통해 가지치기된 모델의 성능을 복구
- **손실 차이**를 기준으로 **샘플링 가중치**를 동적으로 업데이트 -> 더 많은 학습이 필요한 도메인에 데이터를 집중적으로 샘플링
- **효율적인 데이터 샘플링**을 통해 **훈련 속도**를 개선하고, 모델이 더 잘 학습할 수 있도록 함

**핵심 아이디어:** 모델이 각 데이터 도메인에서 '어려워하는 정도'를 측정하고, 더 어려워하는 도메인의 데이터를 더 많이 샘플링하여 학습 효율을 극대화

↑  
이 '어려워하는 정도'를 측정하기 위한 기준점이 '참조 손실(Reference Loss)'

---



# Dynamic Batch Loading 설명

$\ell_{ref}[i]$  참조 손실( $\ell_{ref}$ ) 이란, 만약 이 목표 모델(예: 1.3B)을 처음부터 제대로 학습했다면 각 도메인에서 달성했어야 할 ‘이상적인 손실 값’  
따라서, 이 값을 알면 현재 모델의 손실과 비교하여 얼마나 더 학습이 필요한지 알 수 있음

영.. 그걸 어떻게 아는데..?

바로 스케일링 법칙(Scaling Laws) 을 사용하여 예측  
모델 크기, 데이터셋 크기, 컴퓨팅 양에 따른 모델 성능(손실)의 변화를 예측하는 식

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

Scaling Law 수식

$\ell_{ref}[i]$

| 기호                   | 의미                                                                 | 항목                    | 의미                                                    |
|----------------------|--------------------------------------------------------------------|-----------------------|-------------------------------------------------------|
| $L(N, D)$            | 모델의 예상 손실 (loss) — 모델 크기 $N$ 과 데이터 크기 $D$ 에 따라 결정됨                 |                       |                                                       |
| $E$                  | 이상적인 손실값 (asymptotic loss): 무한히 큰 모델과 무한 데이터가 있을 때 이론적으로 도달 가능한 손실 | $L(N, D)$             | 손실(loss): 모델의 예측 성능 지표 (낮을수록 좋음)                      |
| $\frac{A}{N^\alpha}$ | **모델 크기 $N$ **가 커질수록 손실이 감소하는 항 — $\alpha > 0$                     | $N$                   | <b>모델 파라미터 수</b> = 모델 크기 (예: 1.3B = 13억 개 파라미터)       |
| $\frac{B}{D^\beta}$  | **데이터 크기 $D$ **가 많을수록 손실이 줄어드는 항 — $\beta > 0$                     | $D$                   | <b>훈련에 사용된 토큰 수</b> (예: 300B tokens = 3000억 개 텍스트 단위) |
| $A, B$               | 스케일 조절을 위한 상수 (도메인별로 다름)                                           | $E$                   | 이론적인 최소 손실 (loss floor)                               |
| $\alpha, \beta$      | 각각 모델 크기와 데이터 크기의 영향력(민감도) 계수                                      | $A, B, \alpha, \beta$ | 실험적으로 추정되는 스케일링 계수 (도메인마다 다름)                         |

- 
- LLaMA2-1.3B처럼 아직 존재하지 않거나 학습되지 않은 모델이 특정 도메인에서 훈련된다면 이론적으로 도달할 수 있는 손실을 추정하는 과정

논문에서 사용하는 \*\*LLaMA2 모델들(7B, 13B, 70B)\*\*은 모두 동일한 2T 토큰으로 학습됨.

이 경우 데이터 크기 D는 고정값(D=2T)이고, 변수는 모델 크기 N

$$L(N) = E' + \frac{A}{N^\alpha} \quad (\text{where } E' = E + \frac{B}{D^\beta} \text{ is 상수}) \quad \text{단순화 한 Scaling Law 수식}$$

논문에서는 세 모델(예: LLaMA2-7B, 13B, 70B)의 손실 값으로 A,  $\alpha$ , E' 를 피팅할 수 있다고 함

만약 모델들이 서로 다른 데이터 크기로 학습되었다면 E, A, B,  $\alpha$ ,  $\beta$  총 5개 변수를 추정하기 위해 더 많은 모델의 손실 값이 필요(수렴을 위함)

| 모델         | 토큰 수(data) | 손실   |
|------------|------------|------|
| LLaMA2-7B  | 100B       | 2.10 |
| LLaMA2-13B | 100B       | 1.95 |
| LLaMA2-70B | 100B       | 1.80 |

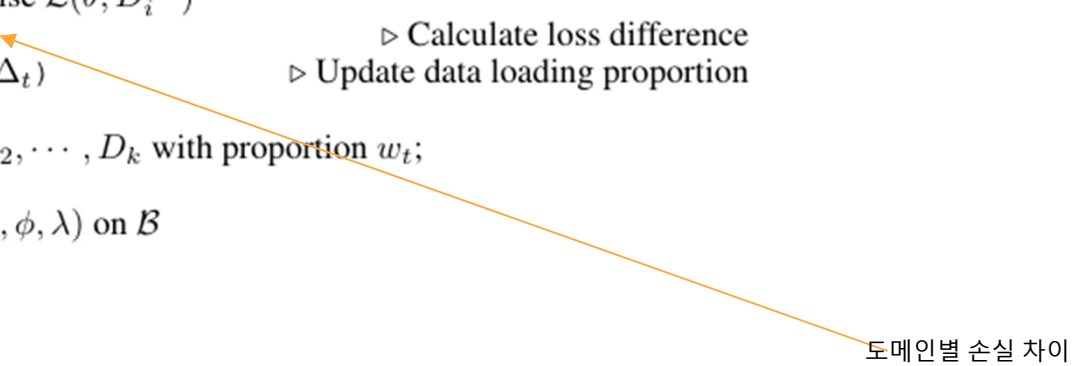
---

**Algorithm 1:** Dynamic Batch Loading

---

**Require:** Training data of  $k$  domains  $D_1, D_2, \dots, D_k$ , validation data  $D_1^{\text{val}}, D_2^{\text{val}}, \dots, D_k^{\text{val}}$ , initial data loading weights  $w_0 \in \mathbb{R}^k$ , reference loss  $\ell_{\text{ref}} \in \mathbb{R}^k$ , LM loss  $\mathcal{L}$  or pruning loss  $\mathcal{L}_{\text{prune}}$ , training steps  $T$ , evaluation per  $m$  steps, model parameters  $\theta$  ( $\theta, z, \phi, \lambda$  for pruning)

```
for  $t = 1, \dots, T$  do
  if  $t \bmod m = 0$  then
     $\ell_t[i] \leftarrow \mathcal{L}(\theta, z, D_i^{\text{val}})$  if pruning else  $\mathcal{L}(\theta, D_i^{\text{val}})$ 
     $\Delta_t[i] \leftarrow \max\{\ell_t[i] - \ell_{\text{ref}}[i], 0\}$  ▷ Calculate loss difference
     $w_t \leftarrow \text{UpdateWeight}(w_{t-m}, \Delta_t)$  ▷ Update data loading proportion
  end
  Sample a batch of data  $\mathcal{B}$  from  $D_1, D_2, \dots, D_k$  with proportion  $w_t$ ;
  if pruning then
    Update  $\theta, z, \phi, \lambda$  with  $\mathcal{L}_{\text{prune}}(\theta, z, \phi, \lambda)$  on  $\mathcal{B}$ 
  else
    Update  $\theta$  with  $\mathcal{L}(\theta, \mathcal{B})$ 
  end
end
```



도메인별 손실 차이

**Subroutine**  $\text{UpdateWeight}(w, \Delta)$

```
   $\alpha \leftarrow w \cdot \exp(\Delta)$  ▷ Calculate the unnormalized weights
   $w \leftarrow \frac{\alpha}{\sum_i \alpha[i]}$  ▷ Renormalize the data loading proportion
  return  $\theta$ 
```

---

masks often converge fast. Therefore, we only allocate a limited budget for pruning (see Table 5). Following pruning, we finalize the pruned architecture by preserving the highest-scoring components associated with the mask variables in each substructure, and continue pre-training the pruned model with the language modeling objective. We refer to this second stage as continued pre-training.

---

## 실험 및 결과

### 가지치기

Table 4: Validation perplexity and generation speed during inference (tokens/second) of targeted structured pruning with a uniform layer configuration, and CoFiPruning, with a non-uniform layer configuration. Inference speed is measured on a Nvidia A100 (80G) GPU, on a singal instance generating up to 512 tokens.

|      | Layer Config     | PPL ↓ | Speed ↑ |      | Layer Config     | PPL ↓ | Speed ↑ |
|------|------------------|-------|---------|------|------------------|-------|---------|
| 1.3B | CoFiPruning      | 9.1   | 51      | 2.7B | CoFiPruning      | 7.0   | 37      |
|      | Targeted pruning | 10.3  | 58      |      | Targeted pruning | 7.7   | 43      |

---

## 실험 및 결과

### 동적 배치

Table 3: Domain data usage with dynamic batch loading compared to the original proportions.

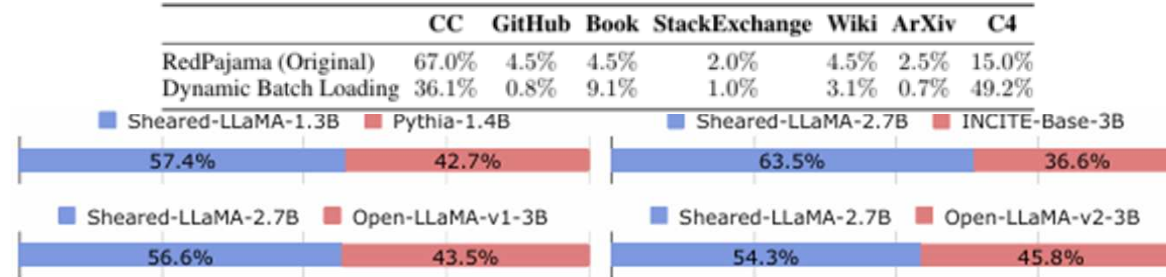


Figure 3: Sheared-LLaMAs outperform Pythia-1.4B, INCITE-Base-3B, OpenLLaMA-3B-v1 and OpenLLaMA-3B-v2 in instruction tuning.

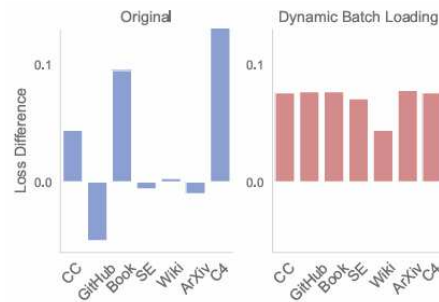


Figure 4: Loss difference between the pruned model (1.3B) and estimated reference loss, with original vs. dynamic batch loading.

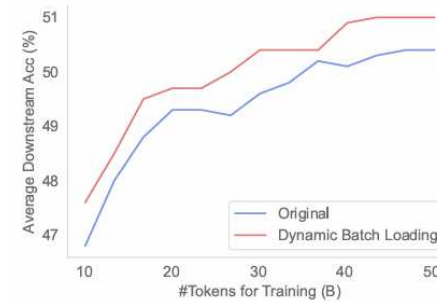


Figure 5: Downstream task performance of Sheared-LLaMA-1.3B with original data proportion and dynamic batch loading.

Table 2: Sheared-LLaMA outperforms publicly available models of comparable size on downstream tasks. The shot number used is noted in parentheses, with 0-shot if not specified. Models with † use a different training data from RedPajama. Please refer to Table 1 for details.

| Model (#tokens for training) | Commonsense & Reading Comprehension |             |             |                 |             |                |
|------------------------------|-------------------------------------|-------------|-------------|-----------------|-------------|----------------|
|                              | SciQ                                | PIQA        | WinoGrande  | ARC-E           | ARC-C (25)  | HellaSwag (10) |
| LLaMA2-7B (2T)†              | 93.7                                | 78.1        | 69.3        | 76.4            | 53.0        | 78.6           |
| OPT-1.3B (300B)†             | 84.3                                | 71.7        | <b>59.6</b> | 57.0            | 29.7        | 54.5           |
| Pythia-1.4B (300B)†          | 86.4                                | 70.9        | 57.4        | 60.7            | 31.2        | 53.0           |
| TinyLlama-1.1B (3T)†         | <b>88.9</b>                         | 73.3        | 58.8        | 55.3            | 30.1        | 60.3           |
| Sheared-LLaMA-1.3B (50B)     | 87.3                                | <b>73.4</b> | 57.9        | <b>61.5</b>     | <b>33.5</b> | <b>60.7</b>    |
| OPT-2.7B (300B)†             | 85.8                                | 73.7        | 60.8        | 60.8            | 34.0        | 61.5           |
| Pythia-2.8B (300B)†          | 88.3                                | 74.0        | 59.7        | 64.4            | 36.4        | 60.8           |
| INCITE-Base-3B (800B)        | 90.7                                | 74.6        | 63.5        | <b>67.7</b>     | 40.2        | 64.8           |
| Open-LLaMA-3B-v1 (1T)        | 91.3                                | 73.7        | 61.5        | 67.6            | 39.6        | 62.6           |
| Open-LLaMA-3B-v2 (1T)†       | <b>91.8</b>                         | <b>76.2</b> | 63.5        | 66.5            | 39.0        | 67.6           |
| Sheared-LLaMA-2.7B (50B)     | 90.8                                | 75.8        | <b>64.2</b> | 67.0            | <b>41.2</b> | <b>70.8</b>    |
| Model (#tokens for training) | Continued                           |             | LM          | World Knowledge |             | Average        |
|                              | LogiQA                              | BoolQ (32)  | LAMBADA     | NQ (32)         | MMLU (5)    |                |
| LLaMA2-7B (2T)†              | 30.7                                | 82.1        | 28.8        | 73.9            | 46.6        | 64.6           |
| OPT-1.3B (300B)†             | 26.9                                | 57.5        | 58.0        | 6.9             | 24.7        | 48.2           |
| Pythia-1.4B (300B)†          | <b>27.3</b>                         | 57.4        | <b>61.6</b> | 6.2             | <b>25.7</b> | 48.9           |
| TinyLlama-1.1B (3T)†         | 26.3                                | 60.9        | 58.8        | <b>12.1</b>     | 25.5        | 50.0           |
| Sheared-LLaMA-1.3B (50B)     | 26.9                                | <b>64.0</b> | 61.0        | 9.6             | <b>25.7</b> | <b>51.0</b>    |
| OPT-2.7B (300B)†             | 26.0                                | 63.4        | 63.6        | 10.1            | 25.9        | 51.4           |
| Pythia-2.8B (300B)†          | 28.0                                | 66.0        | 64.7        | 9.0             | 26.9        | 52.5           |
| INCITE-Base-3B (800B)        | 27.7                                | 65.9        | 65.3        | 14.9            | <b>27.0</b> | 54.7           |
| Open-LLaMA-3B-v1 (1T)        | 28.4                                | 70.0        | 65.4        | <b>18.6</b>     | <b>27.0</b> | 55.1           |
| Open-LLaMA-3B-v2 (1T)†       | 28.1                                | 69.6        | 66.5        | 17.1            | 26.9        | 55.7           |
| Sheared-LLaMA-2.7B (50B)     | <b>28.9</b>                         | <b>73.7</b> | <b>68.4</b> | 16.5            | 26.4        | <b>56.7</b>    |



Q&A

